Four weather APIs, one postcode, 5.7 degrees apart

Same coordinates, same minute, a 5.7°C spread in reported current temperature. One provider reported zero precipitation while the others reported rain. The disagreement was the finding; the harder problem was what to do with it.

16 AUG 2026 · 8 MIN READ

On 30 May 2026, four weather APIs were queried for the same German postcode, PLZ 93125, within the same minute. They disagreed about the current temperature by 5.7°C. They disagreed about whether it was raining. MET.no alone reported zero precipitation while the others did not.

This was not a forecast comparison. Forecasts are expected to diverge, and nobody is surprised when two models disagree about Thursday. This was *current conditions*, the field most applications treat as ground truth, sampled simultaneously for one point on the map.

Why the disagreement mattered here

The consumer was a ride-timing advisor for a motorcycle. Its job is to answer one question: is now a good time to go out.

The dangerous case is not heavy rain. Heavy rain is obvious from a window. The dangerous case is the first rain after a dry spell, when accumulated oil and rubber on the road surface lifts and the grip drops sharply for roughly the first twenty minutes. Getting that call wrong in the optimistic direction has a physical cost, which is a useful property in a test system: it removes the temptation to round in the comfortable direction.

A tool that answers this from a single provider inherits that provider's blind spot completely, and inherits it invisibly. The API returns a number with no error bar and no indication that three other reputable services would have answered differently.

What was measured

Four providers, queried directly rather than through a wrapper, for one postcode, on one date, for current conditions. Plus DWD radar, the German national weather service's precipitation radar, at roughly 5-minute and 1-kilometre resolution.

MEASURE	RESULT
Providers compared	4
Location	PLZ 93125, Germany
Spread in reported current temperature	5.7°C
Providers reporting zero precipitation	1 of 4 (MET.no)
Radar resolution used instead	~5 min, ~1 km (DWD)
Date of benchmark	30 May 2026

A 5.7°C spread is not a rounding difference or a units bug. It is the difference between a road surface at 4°C and a road surface at 10°C, which for a motorcycle is the difference between two genuinely different decisions.

Why providers disagree about the present tense

The word "current" is doing a lot of unearned work in these APIs. Depending on the provider, it can mean the nearest physical station reading, an interpolation between several stations, or the zeroth hour of a model run that was computed some time ago. Those are three different quantities wearing the same field name.

Station density explains much of the temperature spread. Interpolating between distant stations across changing terrain produces a plausible number with no honest confidence attached. The precipitation disagreement is different and more interesting: light rain at the edge of detection is genuinely a threshold call, and providers place that threshold differently. A provider reporting zero is not necessarily wrong. It is answering a slightly different question.

What replaced the single-source read

Radar for the thing that matters, model data for context. Precipitation is observed rather than inferred, so DWD radar at 5-minute and 1-kilometre resolution answers "is water falling here, now" far better than any provider's current-conditions field. Temperature stays fused across sources, because there is no equivalent direct observation available for an arbitrary point.

The rule that came out of it: prefer the source that observed the thing over the source that modelled it, and treat provider disagreement as a signal in its own right rather than as noise to average away. When four services disagree sharply, that is information about confidence, and collapsing it to a mean throws away the only honest thing in the dataset.

The specific fusion weights are not published, because they are tuned to one region's station density and would be actively misleading applied elsewhere.

What got built wrong first

Fusing the sources was the easy half. Acting on the result was where the real mistake happened, and it took until 25 June to admit it.

The first implementation was a hardcoded safety net in the server. When a conversation mentioned riding and the weather conditions matched a rule, it appended a verbatim warning, in capitals, about oil-film grip risk. The engineering was fine. The behaviour was terrible.

It produced contradictory messages, because a hardcoded block injected after the fact does not know what the rest of the response already said. It fired on the word "ride" regardless of context, so mentioning a past ride triggered a warning about a ride nobody was planning. And an all-caps system alert stapled onto an otherwise normal answer reads as a machine malfunctioning, which is the fastest way to teach someone to ignore a safety warning entirely.

The whole thing was removed on 25 June, including the weather fetch and the warning injection. Not softened. Deleted. A safety feature that gets ignored is worse than no safety feature, because it consumes the attention budget that a real warning would need.

What is still unknown

- Whether the 5.7°C spread is typical or was an unusually bad day. It was one benchmark on one date at one location. A defensible number would need repeated sampling across weeks and several postcodes.
- Which provider was actually right. There was no reference thermometer at the location, so the benchmark establishes disagreement, not accuracy. That distinction gets skipped often and it matters.
- Whether radar-first genuinely improves the oil-film call, or only improves confidence in it. The outcome is unmeasurable without deliberately riding in conditions the tool advises against.

Where this generalises

Any product that reads a third-party API and renders the number without qualification has this exposure. Pricing feeds, geolocation, air quality, traffic, currency rates, address validation. The failure mode is not an outage, which is visible and gets handled. It is a confidently wrong value returned with a 200, formatted correctly, and shown to a user who has no way to know that a second provider would have said something else.

The cheap diagnostic is to query a second source once, by hand, and compare. It takes an afternoon at most. In this case the answer was 5.7 degrees, and the architecture changed because of it.

Fabi writes about backend systems, data pipelines and the protocols underneath them. More at **github.com/Fabi-SPL/findings**.
Originally published at https://lucidailabs.com/essays-four-weather-apis.
© 2026 Fabian Ilg. Licensed CC BY-NC-ND 4.0. Share it with attribution, do not sell it, do not publish altered versions.